

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«АЛТАЙСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных технологий
Кафедра информатики

**Разработка backend-части информационной системы управления
мероприятиями**
выпускная квалификационная работа

Выполнил:
студент группы 4.205-2,
Шацких Алексей Евгеньевич

(подпись)

Научный руководитель:
к.т.н., доцент
Михеева Татьяна Викторовна

(подпись)

Работа защищена:
«__» _____ 2026 г.

Оценка: _____

Председатель ГЭК:
д.т.н., профессор
Леонов Сергей Леонидович

(подпись)

Допустить к защите:
Зав. кафедрой, к.ф.-м.н., доцент
Козлов Денис Юрьевич

(подпись)

«__» _____ 2026 г.

Барнаул 2026

РЕФЕРАТ

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ BACKEND РАЗРАБОТКИ КЛИЕНТ- СЕРВЕРНЫХ ПРИЛОЖЕНИЙ	6
1.1. Методологии и подходы backend-разработки	6
1.2. Архитектурные подходы backend разработки	14
1.3. Анализ и выбор программных средств разработки	25
2. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ	26
2.1. Расчёт экономической выгоды	26
2.2. Проектирование базы данных	26
3. РАЗРАБОТКА BACKEND-ЧАСТИ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ	27
3.1. Разработка доменного слоя	27
3.2. Разработка инфраструктурного слоя	27
3.3. Разработка слоя бизнес-логики	27
3.4. Разработка слоя API	27
ЗАКЛЮЧЕНИЕ	28
БИБЛИОГРАФИЧЕСКИЙ СПИСОК	29
ПРИЛОЖЕНИЕ	32

ВВЕДЕНИЕ

Мероприятия различного вида проводятся каждый день в миллионах организаций по всему миру. Для проведения мероприятия организующему необходимо выбрать место, создать анонсы в мессенджерах и социальных сетях, узнать, какое количество человек планирует посетить его, возможно, создать таблицу, чтобы участники записывались в неё. Из-за того, что таких мероприятий в крупных организациях проводится большое количество, у организаторов возникают трудности в информационной поддержке организационных процессов, чтобы мероприятие состоялось.

Для решения этих проблем мы предлагаем создать информационную систему мероприятий, которая была бы встраиваемой в уже существующую экосистему организации, и позволяющую контролировать все процессы организации мероприятий в одном месте.

Актуальность темы обусловлена растущей цифровизацией всех отраслей экономики, что влечёт за собой увеличение потребности рынка в новых цифровых решениях привычных и рутинных задач.

Разработка предлагаемой информационной системы требует применения современных подходов и технологий, таких как использование паттернов проектирования, фреймворков, технологий тестирования и баз данных.

Данная выпускная квалификационная работа посвящена разработке backend-части системы, которая является ключевой составляющей всей системы, обеспечивая её стабильное функционирование, обработку бизнес-логики, хранение данных и интеграцию с внешними сервисами.

Целью данной выпускной квалификационной работы является разработка backend-части информационной системы.

Для достижения поставленной цели необходимо решить следующие **задачи:**

1. Изучить основные методы и технологии backend-разработки.
2. Выбрать архитектурные принципы и паттерны проектирования.
3. Выбрать программные средства разработки.
4. Спроектировать базу данных.
5. Разработать доменный слой.
6. Разработать инфраструктурный слой.
7. Разработать слой бизнес-логики.
8. Разработать слой API.

Объектом исследования данной работы является методология и практика разработки клиент-серверных приложений.

Предмет исследования – backend разработка с применением фреймворка ASP.NET.

Практическая значимость разработки backend-части системы управления мероприятиями заключается в создании поддерживаемой, расширяемой, масштабируемой и отказоустойчивой инфраструктуры системы.

Разрабатываемая система позволит упростить информационную поддержку процессов проведения мероприятий, что позволит увеличить вовлечённость участников.

1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ BACKEND РАЗРАБОТКИ КЛИЕНТ-СЕРВЕРНЫХ ПРИЛОЖЕНИЙ

1.1. Методологии и подходы backend-разработки

При разработке backend-приложений, как и при разработке любого программного обеспечения, применяются различные методологии и подходы, позволяющие упростить разработку, поддержку, уменьшить количество ошибок в программном коде, а также увеличить взаимопонимание между заказчиком и командой разработки. Рассмотрим некоторые из них.

Test-Driven Development (TDD) [1] – методология, основополагающий принцип которого заключается в разработке программного обеспечения через тестирование. Процесс разработки по данной методологии происходит через небольшие итерации, проходящие в три этапа.

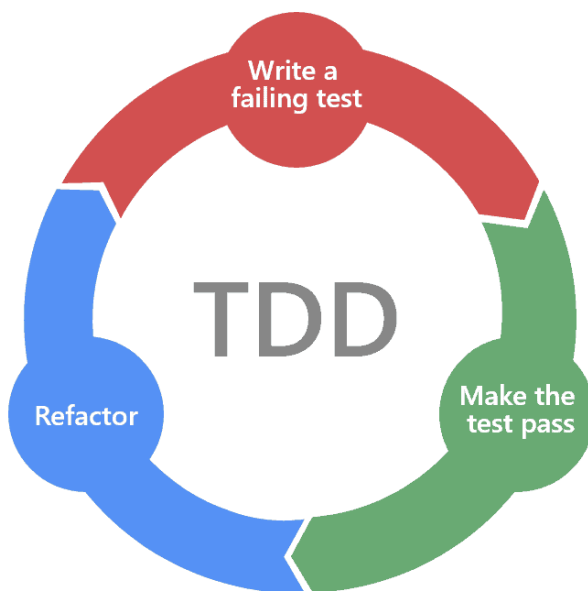


Рис. 1. Test-Driven Development

Первым этапом, именуемым «красной зоной», является написание тестов для ещё не существующего функционала, которые будут выдавать ошибку до того момента, пока не будет написана реализация.

Вторым этапом, именуемым «зелёной зоной», является написание реализации функционала, который будет проходить тесты без ошибок. Данный этап самый короткий и, если функционал проходит тесты, то он считается реализованным.

Третьим этапом, именуемым «синей зоной», является рефакторинг, то есть изменение и улучшение уже существующего функционала и тестов. В данной методологии рефакторинг является безопасным процессом, поскольку ошибки, возникающие при тестировании, содержат в себе точную информацию о месте и причине возникновения.

К достоинствам методологии относится создание тестируемого и легко поддерживаемого кода: поскольку тесты пишутся до реализации, разработчик вынужден проектировать модули с минимальной связностью, что улучшает архитектуру системы. Непрерывно пополняемый набор регрессионных тестов позволяет обнаруживать нарушения существующей функциональности сразу после внесения изменений. Помимо этого, тесты выполняют роль живой документации, точно описывая ожидаемое поведение каждого компонента.

К недостаткам относится замедление разработки на начальном этапе, пока команда не выработала устойчивый навык работы по данной методологии. Дополнительную сложность представляет тестирование кода, взаимодействующего с внешними системами, что требует применения макетов (mock-объектов). При нестабильных требованиях тесты нередко нуждаются в переработке наравне с реализацией, что может нивелировать часть выигрыша от раннего обнаружения ошибок.

Domain-Driven Design (DDD) [2] – это подход проектирования объектов программного обеспечения как сущностей реального мира, с их бизнес-

логикой и жизненным циклом, отсюда проистекает определение данного подхода как «предметно-ориентированное программирование». Рассмотрим основные правила, которые предлагает данный подход.

Первым правилом, которое предлагает DDD, является разделение объектов предметной области на сущности и объекты-значения.

Объекты-значения (value-objects) – это маленькие составные блоки предметной области, которые не являются уникальными, не имеют своей бизнес-логики, жизненного цикла и могут сравниваться по значению. Объекты-значения предполагаются неизменяемыми, поэтому вместо изменения уже существующего объекта класса предлагается создавать новый объект. Рекомендуется, чтобы объекты-значения ссылались только на другие объекты-значения посредством объектов классов.

Сущности (entities) – это объекты предметной области, которые составлены из объектов-значений, они имеют свою бизнес-логику, жизненный цикл и являются уникальными, поэтому для их однозначного определения вводится идентификатор, который становится операндом сравнения. Сущности предполагаются изменяемыми, но рекомендуется инкапсулировать логику их изменения в публичных методах, не допуская прямого изменения полей объекта класса. Также рекомендуется, чтобы сущности ссылались на объекты-значения и другие сущности посредством объектов классов.

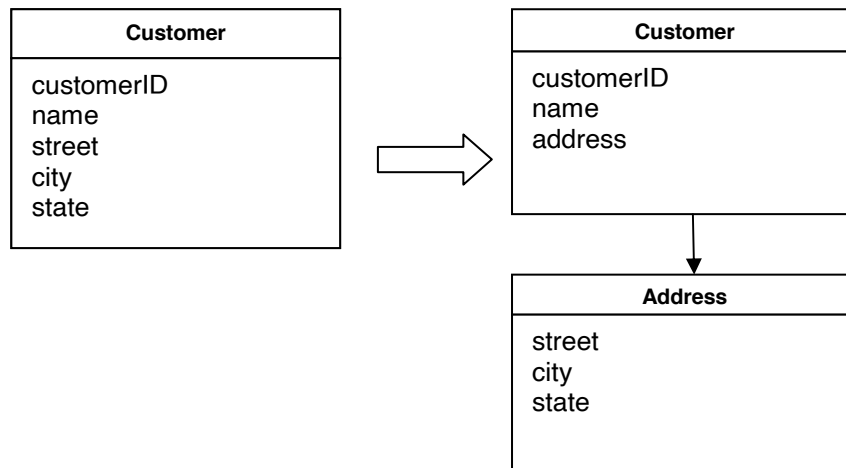


Рис. 2. Пример использования сущностей и объектов-значений

Вторым правилом, которое предлагает DDD, является выделение связанных между собой сущностей и объектов-значений в отдельные кластеры, называемые агрегатами. В каждом агрегате предлагается выделить главенствующую сущность, называемую корнем агрегата (*aggregate root*), которая будет отвечать за управление связанными сущностями и объектами-значениями. Рекомендуется, чтобы корень агрегата ссылался на другие корни агрегатов через идентификаторы.

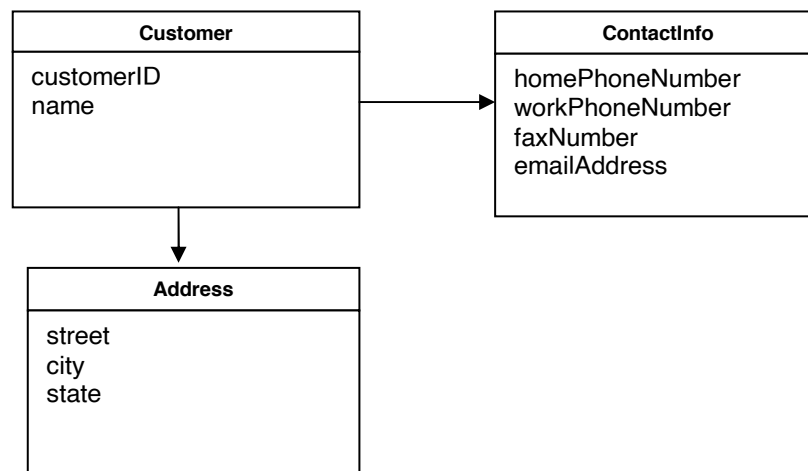


Рис. 3. Пример агрегата Customer

Третьим правилом, которое предлагает DDD, является выделение логики создания связанных сущностей в отдельные фабрики, что позволяет решить проблему разграничения зон ответственности сущностей.

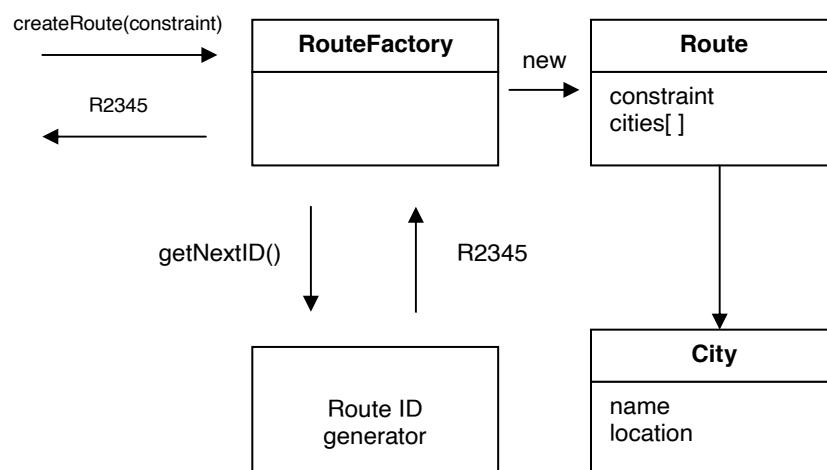


Рис. 4. Пример фабрики Route

Четвёртым правилом, которое предлагает DDD, является привнесение в доменную область репозитория (repositories) – объектов, которые отвечают за сохранение сущности в программном обеспечении. Как правило, в доменной области определяется только интерфейс репозитория, а непосредственно реализация этого интерфейса происходит в инфраструктурном слое приложения.

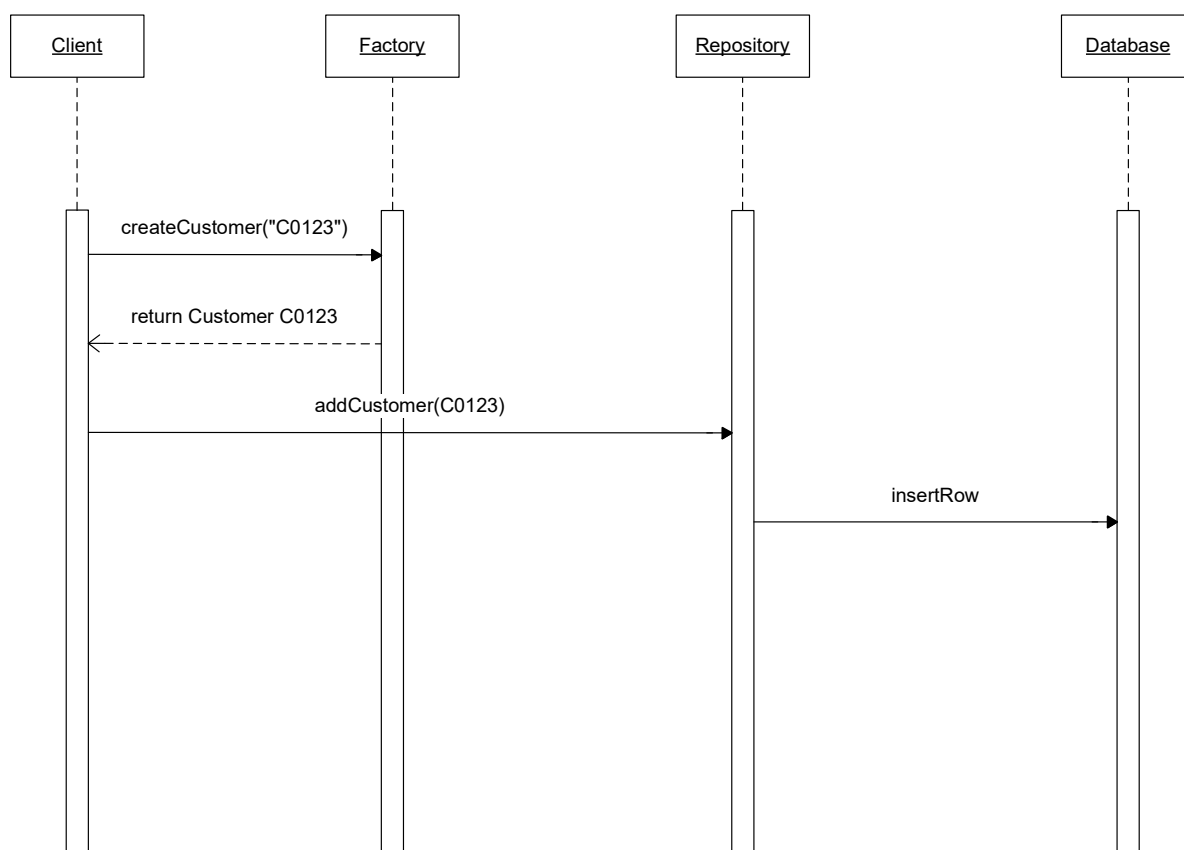


Рис. 5. Пример логики работы с репозиторием Customer

К достоинствам данного подхода относится инкапсуляция бизнес-логики в рамках единой доменной модели, что упрощает её повторное использование и перенос в между проектами, упрощение проектирования для разработчиков, не являющихся экспертами в предметной области, увеличение понимания между заказчиком и командой разработки, посредством вырабатывания единого языка (Ubiquitous Language).

К недостаткам данного подхода относится необходимость высокой квалификации разработчиков для её эффективного применения, неприменимость без соответствующих архитектурных решений, что влечёт за собой увеличение затрат на первоначальное проектирование и увеличение итоговой стоимости проекта. По этим причинам DDD подходит проектам со сложной и долгоживущей бизнес-логикой, тогда как для небольших и краткосрочных проектов затраты на применение подхода, как правило, не окупаются.

Behavior-Driven Development (BDD) [3] – подход к разработке программного обеспечения, являющийся эволюцией метода Test-Driven Development. Основной принцип подхода состоит в том, что до написания какой-либо реализации специалисты предметной области совместно с разработчиками и тестировщиками описывают желаемое поведение системы на структурированном естественном языке. Наиболее распространённым форматом такого описания является конструкция Given-When-Then: предусловие (Given) задаёт начальное состояние системы, событие (When) описывает действие пользователя, а ожидаемый результат (Then) фиксирует наблюдаемое поведение, которое должна продемонстрировать система. Полученные сценарии преобразуются в автоматические тесты с помощью

специализированных инструментов, таких как Cucumber [4], SpecFlow [5] или Behave [6], и служат одновременно исполняемой спецификацией и приёмочными критериями.

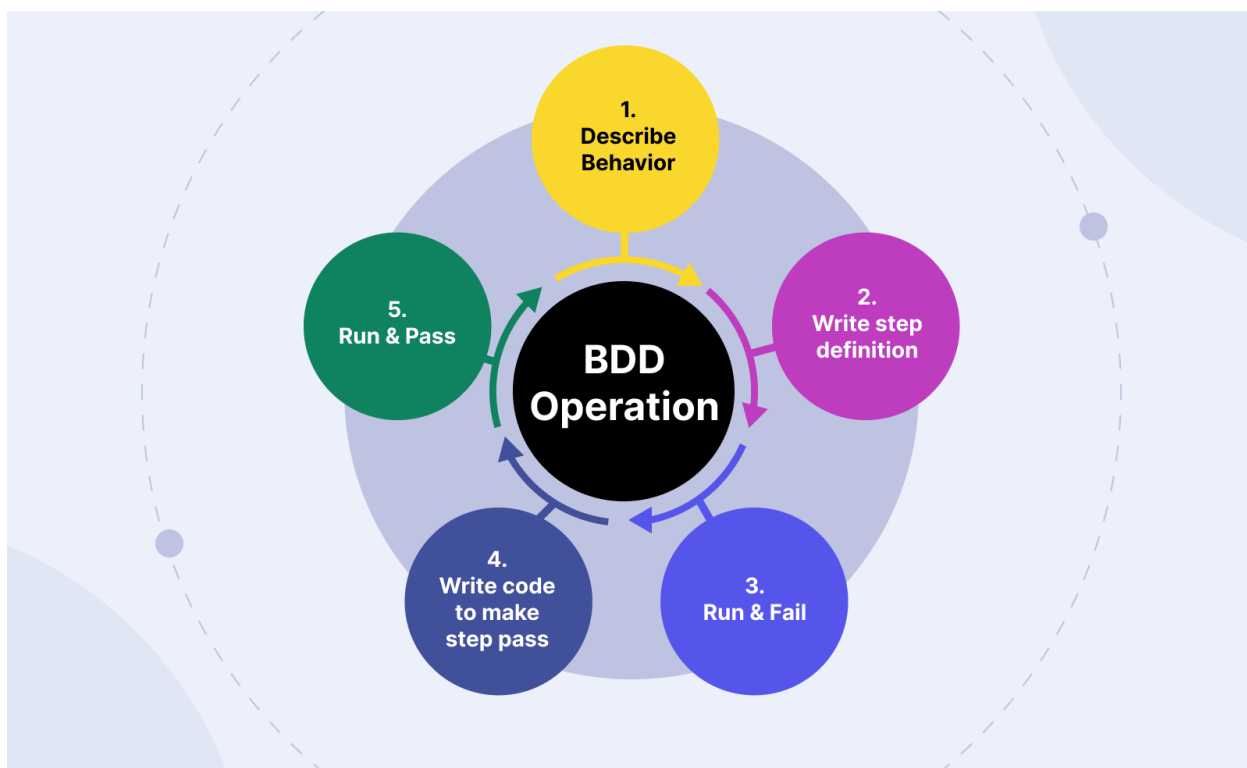


Рис. 6. Behavior-Driven Development

Подход решает ключевое ограничение TDD, при котором тесты остаются артефактом, понятным только разработчикам. В BDD сценарии становятся общим языком между заказчиком, аналитиком, тестировщиком и разработчиком, что снижает риск расхождения между реализованной функциональностью и бизнес-требованиями. Это достигается за счёт непосредственного участия представителей предметной области в формулировке сценариев на ранних стадиях разработки.

Вместе с тем применение BDD сопряжено с рядом издержек. Поддержание актуальности сценариев требует постоянных усилий при изменении требований. Внедрение подхода предполагает обучение всех участников процесса работе с предметно-ориентированным языком и

инструментами автоматизации. Совокупность этих факторов делает BDD более ресурсоёмким по сравнению с традиционными подходами, однако данные затраты, как правило, окупаются в проектах с высокой сложностью бизнес-логики и требованиями к долгосрочной сопровождаемости.

Spec-Driven Development (SDD) [7] – относительно новый подход, появившийся на фоне активного развития нейросетей и ИИ-агентов [8]. Суть подхода заключается в том, что перед написанием какой-либо реализации команда проекта определяет требования, поведение, технологии и ограничения программного обеспечения, фиксируя всё это в файлах спецификации. Данные файлы становятся источником истины как для человека, так и для ИИ-агента, что позволяет устранить ключевые системные ограничения при работе с языковыми моделями.

Выделяют четыре основные проблемы, решаемые данным подходом.

Во-первых, ограниченность области и длительности задач: языковые модели демонстрируют деградацию качества при изменениях, затрагивающих более 3-5 файлов одновременно. SDD решает эту проблему посредством принудительной декомпозиции функциональности на атомарные задачи.

Во-вторых, отсутствие контекста о конкретной разрабатываемой функциональности: спецификации обеспечивают ИИ-агенту полное представление о требованиях, граничных случаях, критериях приёмки и точках интеграции ещё до начала генерации кода.

В-третьих, незнание стандартов и соглашений конкретного проекта: специальный файл конституции (`constitution.md`) описывается технологический стек, правила именования, архитектурные решения, допустимые библиотеки и требования безопасности.

В-четвёртых, неконтролируемая автономия ИИ-агента: в рабочий процесс встраиваются обязательные контрольные точки проверки, на которых человек проверяет спецификацию, архитектурный план и декомпозицию задач перед запуском автоматической реализации.

Рабочий процесс SDD включает шесть последовательных этапов: формирование конституции проекта (Constitution), составление спецификации функциональности (Specify), устранение неоднозначностей (Clarify), архитектурное планирование (Plan), декомпозиция на задачи (Tasks) и непосредственная реализация (Implement). Такая структура смещает проектирование и принятие ключевых решений на ранние стадии разработки, сокращая объём переработок и технический долг на поздних этапах.

Для разработки backend-части информационной системы управления мероприятиями был выбран DDD подход, поскольку он упростит моделирование бизнес-логики управления мероприятиями, сделает программный код поддерживаемым для дальнейшего развития, упростит модульное тестирование, а также позволит приобрести необходимые в современных реалиях навыки проектирования, построения архитектуры и использования паттернов.

1.2. Архитектурные подходы backend разработки

При разработке backend-приложений, как и при разработке любого программного обеспечения, применяются различные архитектурные подходы и паттерны, которые позволяют сделать код более читаемым, поддерживаемым, а также снизить порог входа для новых членов команды разработки.

Сперва рассмотрим архитектурные подходы на уровне приложения в целом.

Монолитная архитектура [9] – это подход к разработке программного обеспечения, при котором все модули тесно связаны между собой и представляют неделимое целое. В контексте клиент-серверных приложений монолитная архитектура предполагает размещение клиентского и серверного кода в рамках единого проекта, при этом клиентская часть неотделима от серверной.

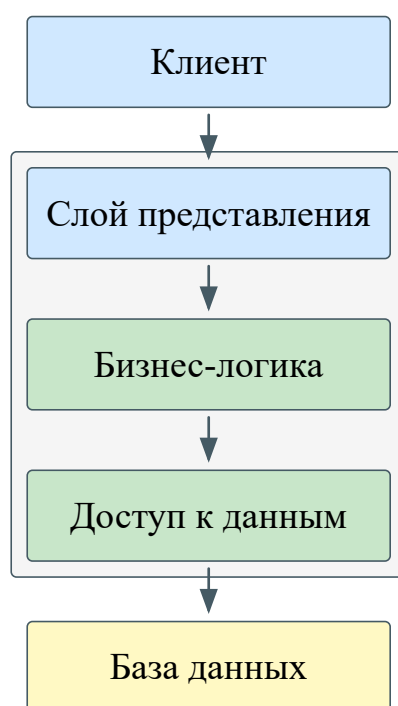


Рис. 7. Монолитная архитектура

К достоинствам монолитной архитектуры относятся простота и скорость разработки, обусловленные сосредоточением всей кодовой базы в одном месте, высокая производительность за счёт выполнения всех операций в рамках единого процесса, а также единая среда хранения данных, устраняющая необходимость в сложных механизмах их синхронизации.

Недостатки монолитной архитектуры обусловлены теми же характеристиками, что и её достоинства. В частности, масштабирование приложения осуществляется только как единого целого, что влечёт за собой избыточное потребление ресурсов. При росте числа пользователей исполнение

всего кода в рамках одного процесса становится узким местом с точки зрения производительности. Помимо этого, внедрение новых технологий может потребовать перепроектирования всего приложения, что существенно замедляет его дальнейшее развитие.

На современном этапе монолитную архитектуру целесообразно рассматривать в качестве архитектурного подхода для небольших проектов с ограниченным жизненным циклом, корпоративных приложений с числом пользователей, не превышающим десятков или сотен человек, а также для проектов со сложной предметной областью.

Микросервисная архитектура [9] – это подход к разработке программного обеспечения, при котором программный код разделяется на отдельные автономные модули. В контексте клиент-серверных приложений микросервисная архитектура может рассматриваться на различных уровнях: отделение клиентской части от серверной, разделение клиентской части на микрофронтенды [10], а серверной – на микросервисы.

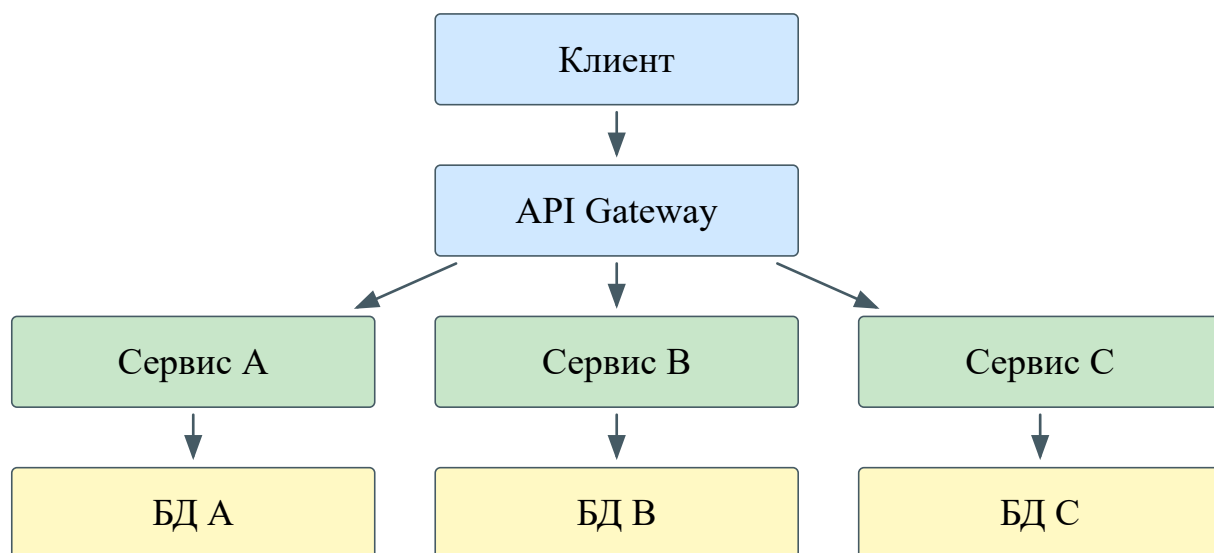


Рис. 8. Микросервисная архитектура

Достоинства микросервисной архитектуры непосредственно обусловлены её основополагающим принципом – разделением программного

кода на автономные модули, что обеспечивает гибкость, масштабируемость, технологическое разнообразие и расширяемость системы.

К недостаткам микросервисной архитектуры относятся проблемы хранения данных и их синхронизация между модулями, для решения которых требуется применение дополнительных программных средств: очереди сообщений, таких как Apache Kafka [11] или RabbitMQ [12], веб-серверов и балансировщиков нагрузки, таких как nginx [13] или Apache HTTP Server [14], а также единого шлюза API (API Gateway), таких как Yandex API Gateway [15] или KrakenD [16]. Помимо этого, создание микросервисов предъявляет высокие требования к квалификации разработчиков, предполагая глубокие знания объектно-ориентированного программирования и практический опыт работы с перечисленными технологиями.

На современном этапе применение микросервисной архитектуры экономически обосновано преимущественно для крупных проектов с простой предметной областью, большим числом пользователей и потребностью в постоянном масштабировании и расширении функциональности. Использование данного подхода характерно для организаций, располагающих собственным штатом разработчиков, в частности, для банков, таких как Альфа-Банк [17] или Т-Банк [18], а также маркетплейсов, например, Ozon [19] или Wildberries [20].

Модульно-монолитная архитектура [21] – это подход к разработке программного обеспечения, призванный сохранить достоинства монолитной архитектуры, такие как простота разработки и согласованность данных, дополнив их преимуществами микросервисной архитектуры: изоляцией модулей и технологическим разнообразием. Основопологающим принципом

данного подхода является логическое разграничение модулей, в отличие от микросервисной архитектуры, где применяется физическое разграничение модулей.

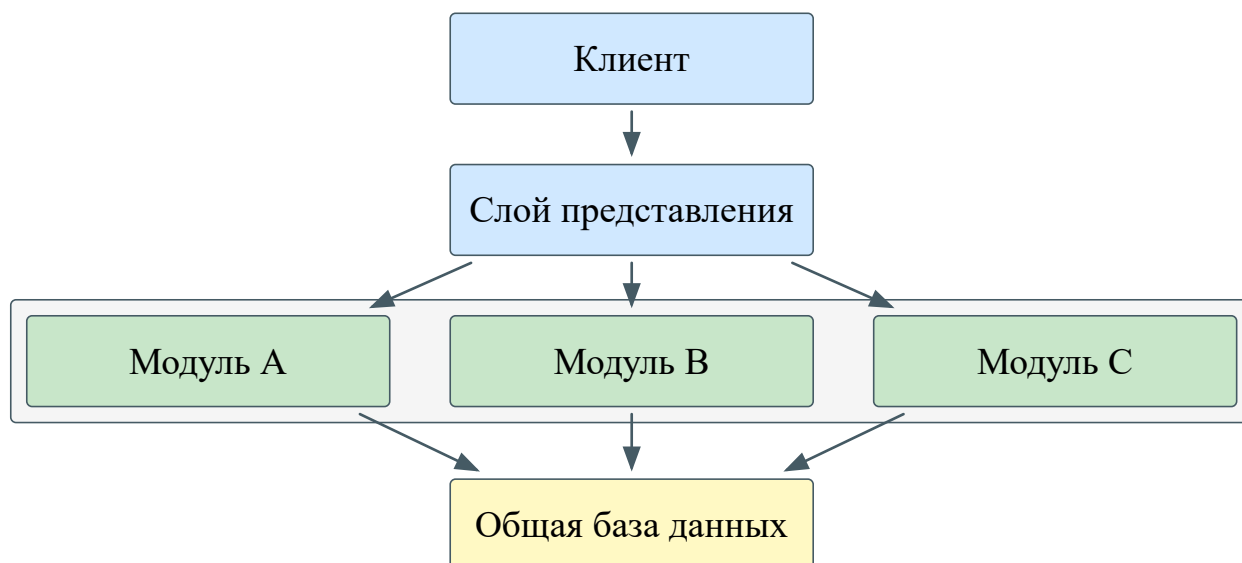


Рис. 9. Модульно-монокитная архитектура

Помимо достоинств, унаследованных от монокитной и микросервисной архитектур, модульный монокит обеспечивает возможность постепенного перехода от монокитной к микросервисной архитектуре а также разграничения зон ответственности при расширении команды или проекта. Кроме того, данный подход снижает требования как к квалификации разработчиков, предъявляемые микросервисной архитектурой, так и к глубине знания предметной области, необходимой при монокитном подходе.

К недостаткам модульного монокита относится сохранение риска возникновения неявных межмодульных зависимостей при недостаточной дисциплине разработки, что способно привести к деградации архитектуры. Развёртывание приложения по-прежнему осуществляется целиком, что исключает возможность независимого обновления или масштабирования отдельных модулей. При значительном росте нагрузки модульный монокит

сохраняет ограничения монолитной архитектуры в части горизонтального масштабирования. Поддержание чётких границ между модулями требует соблюдения архитектурных соглашений внутри команды, что повышает организационные издержки по мере роста проекта.

Рассмотренные архитектурные подходы отвечают на вопрос, как разделить систему на части. Однако вне зависимости от того, какой подход будет выбран, следующим уровнем архитектурных решений становится организация программного кода. Наиболее распространённые способы такой организации рассматриваются далее.

Слоистая архитектура [22] – подход, при котором программный код разделяется на горизонтальные слои, каждый из которых выполняет определённую роль в приложении.

Количество слоёв определяется индивидуально для каждого приложения, однако большинство слоистых архитектур включает четыре стандартных слоёв: слой представления (presentation layer), отвечающий за отображение пользовательского интерфейса и данных, слой бизнес-логики (business layer), отвечающий за логику работы приложения, слой доступа к данным (persistence layer), отвечающий за получение и сохранение данных, и слой абстракции баз данных (database layer), отвечающий за подключение к базам данных. Последние два слоя могут быть объединены в один, особенно когда логика доступа к данным встроена в компоненты бизнес-логики или применяются технологии объектно-реляционного отображения (Object-Relational Mapping, ORM).

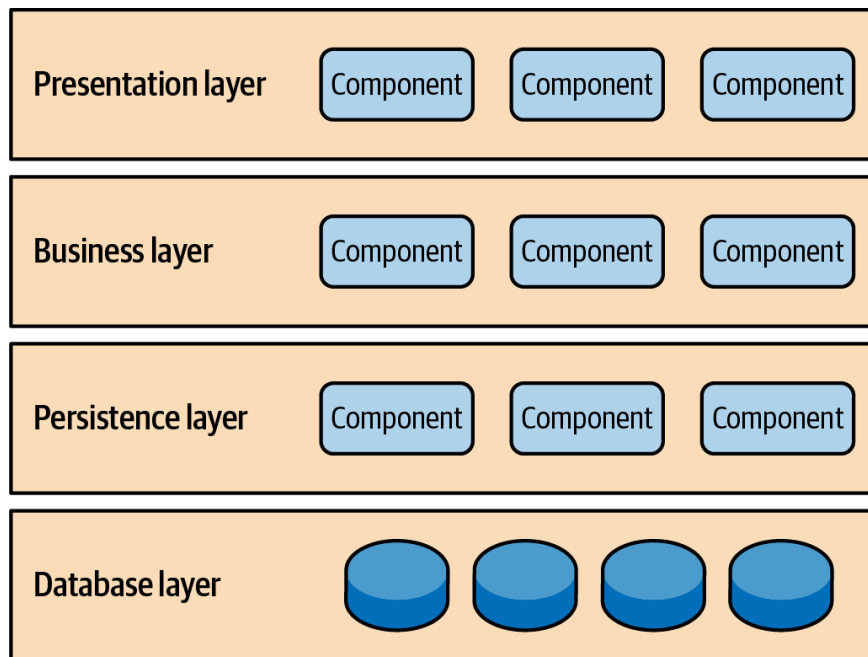


Рис. 10. Слоистая архитектура

Достоинством слоистой архитектуры является разделение ответственности между компонентами, позволяющее инкапсулировать логику каждого слоя и тем самым упростить разработку и тестирование. Ещё одним достоинством слоистой архитектуры выступает концепция закрытых слоёв: запрос, перемещаясь между слоями, обязан проходить через слой, непосредственно расположенный под ним, и не может миновать его.

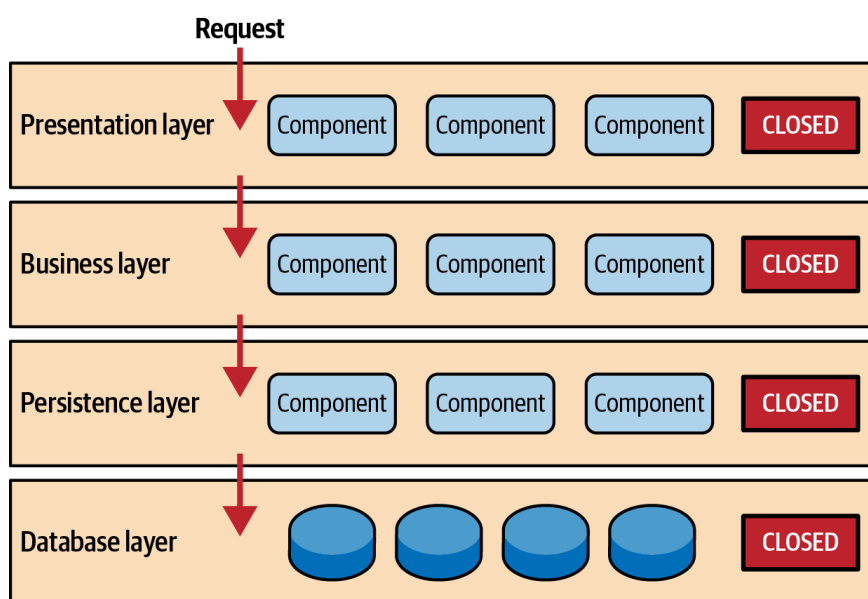


Рис. 11. Концепция закрытых слоёв в слоистой архитектуре

Применение концепции закрытых слоёв позволяет изолировать слои друг от друга, снизить связанность компонентов и вносить изменения в один слой, как правило, не затрагивая соседние.

К недостаткам слоистой архитектуры относят недостаточную изоляцию слоёв: архитектура не предполагает механизмов принудительного соблюдения слоёв, что допускает обращение через слой в обход установленного порядка. Кроме того, зависимость слоя бизнес-логики от слоя доступа к данным затрудняет замену одной СУБД на другую.

Луковая архитектура [23] – подход, при котором программный код организуется в виде концентрических слоёв, направление зависимостей в которых строго ориентировано к центру.

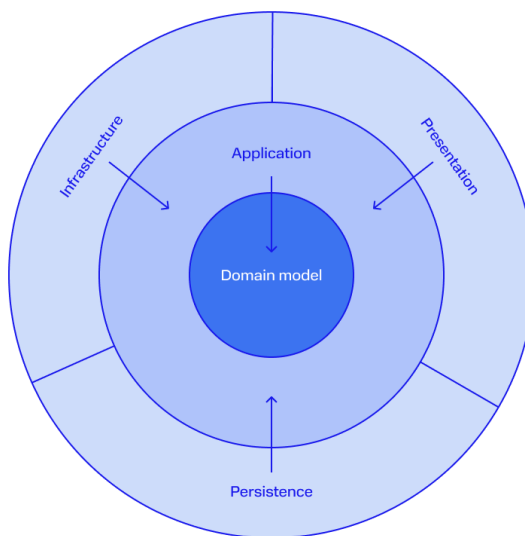


Рис. 12. Луковая архитектура

В центре архитектуры располагается слой предметной области (domain layer), содержащий бизнес-логику и бизнес-правила приложения. Поскольку все зависимости направлены внутрь, слой предметной области не зависит ни от одного из внешних слоёв и связан только с собственными компонентами.

Вокруг слоя предметной области располагается слой приложения (application layer), реализующий бизнес-сценарии (use-cases) и управляющий взаимодействием между слоем предметной области и внешними слоями посредством шлюзов (gateways). Шлюзы представляют собой интерфейсы, объявленные в ядре приложения и транслирующие запросы из модели предметной области во внешний мир и обратно. Конкретные реализации этих интерфейсов выносятся во внешние слои, что обеспечивает изоляцию ядра от деталей инфраструктуры.

Следующим слоем является слой инфраструктуры (infrastructure layer), обеспечивающий взаимодействие приложения с внешними зависимостями: базами данных, файловой системой, веб-сервисами и брокерами сообщений. На этом же уровне располагаются конкретные реализации интерфейсов, объявленных во внутренних слоях. Наконец, внешним слоем является слой представления (presentation layer) – точка входа в приложение, реализующая публичный интерфейс взаимодействия: REST API, подписку на очередь сообщений и т.п.

Ключевым отличием луковой архитектуры от слоистой является отношение к базе данных. В луковой архитектуре база данных не является центром приложения, а выносится на внешний уровень в качестве сменяемой инфраструктурной зависимости. Такой подход смещает фокус проектирования с организации хранения данных на моделирование бизнес-процессов.

Достоинством луковой архитектуры является полная изоляция бизнес-логики от инфраструктурных зависимостей, что позволяет тестировать слой предметной области модульными тестами без необходимости использования базы данных или внешних сервисов. Кроме того, модульная структура

упрощает замену инфраструктурных компонентов, например, переход с одной СУБД на другую, не затрагивая бизнес-логику.

К недостаткам подхода относят повышенную сложность проектирования по сравнению со слоистой архитектурой: разработчику необходимо корректно разграничивать ответственность слоёв и поддерживать строгое соблюдение правила зависимостей, что требует дисциплины и опыта.

Чистая архитектура [24] — подход, предложенный Робертом Мартином в 2012 году и описанный им в одноимённой книге. Чистая архитектура не является принципиально новым решением, а представляет собой синтез ранее предложенных идей: луковой архитектуры, гексагональной архитектуры и подхода Boundary - Control - Entity (BCE). Общей целью всех перечисленных подходов является разделение ответственности посредством разбиения программного кода на слои.

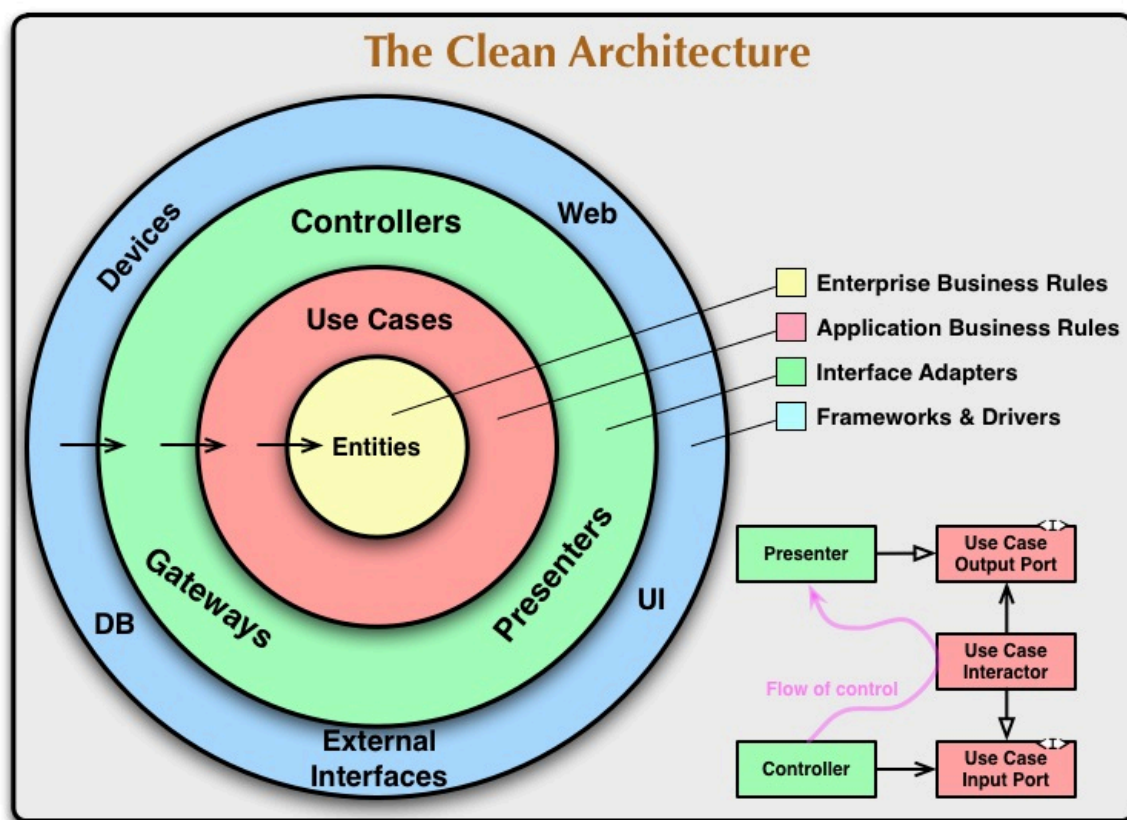


Рис. 13. Чистая архитектура

Центральным принципом чистой архитектуры является правило зависимостей (Dependency Rule): зависимости в исходном коде могут указывать только внутрь, в сторону слоёв с более высоким уровнем абстракции. Ни один компонент внутреннего слоя не должен содержать каких-либо сведений о компонентах внешних слоёв – включая имена классов, функций, переменных и форматы передаваемых данных.

Архитектура включает четыре концентрических слоя. В центре располагается слой сущностей (entities), инкапсулирующий корпоративные бизнес-правила – наиболее стабильную часть системы. Следующим слоем является слой вариантов использования (use cases), содержащий бизнес-правила конкретного приложения и управляющий потоком данных между сущностями. Изменения в этом слое не затрагивают сущности, однако сам слой может изменяться при изменении бизнес-сценариев приложения. Далее следует слой адаптеров интерфейсов (interface adapters), преобразующий данные из формата, удобного для вариантов использования и сущностей, в формат, требуемый внешними системами – базами данных, веб-фреймворками и т.п. Внешним слоем является слой фреймворков и драйверов (frameworks and drivers), содержащий конкретные реализации инфраструктурных зависимостей, именно здесь сосредоточены все детали: веб-сервер, база данных, пользовательский интерфейс.

Ключевым отличием чистой архитектуры от луковой является более детальное разграничение внутренних слоёв: выделение сущностей и вариантов использования в два самостоятельных слоя позволяет явно разделить корпоративные правила, общие для нескольких приложений, и правила,

специфичные для конкретного приложения. В луковой архитектуре это разграничение явно не регламентируется.

Достоинством чистой архитектуры является независимость бизнес-логики от фреймворков, баз данных и пользовательского интерфейса, что обеспечивает возможность полного покрытия бизнес-правил модульными тестами без привлечения инфраструктурных компонентов. Кроме того, изоляция внешних зависимостей упрощает их замену без изменения ядра системы.

К недостаткам подхода относят переусложнение архитектуры на начальном этапе разработки: строгое соблюдение правила зависимостей требует введения дополнительных интерфейсов и объектов передачи данных (data transfer object, DTO) на границах каждого слоя, что увеличивает количество абстракций и усложняет кодовую базу в небольших проектах.

Для разработки backend-части информационной системы управления мероприятиями была выбрана модульно-монолитная архитектура. Данный выбор обусловлен требованием к встраиванию системы в существующую экосистему предприятия: модульная структура обеспечивает достаточную изолированность компонентов для их последующей интеграции со смежными системами, а при необходимости допускает декомпозицию отдельных модулей в самостоятельные микросервисы.

В качестве подхода к внутренней организации программного кода применяется чистая архитектура, обеспечивающая изоляцию бизнес-логики от инфраструктурных зависимостей и возможность независимого тестирования каждого слоя системы.

1.3. Анализ и выбор программных средств разработки

2. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ

2.1. Расчёт экономической выгоды

2.2. Проектирование базы данных

3. РАЗРАБОТКА BACKEND-ЧАСТИ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ

3.1. Разработка доменного слоя

3.2. Разработка инфраструктурного слоя

3.3. Разработка слоя бизнес-логики

3.4. Разработка слоя API

ЗАКЛЮЧЕНИЕ

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Саша Беспоясов. Разработка через тестирование (TDD) [Электронный ресурс]. URL: <https://doka.guide/tools/tdd/> (дата обращения: 15.02.2026).
2. Abel Avram, Floyd Marinescu. Domain Driven Design Quickly. C4Media, 2006.
3. Смарт Джон Фергюсон, Молак Ян. BDD в действии. ДМК-Пресс, 2023.
4. Официальная документация Cucumber [Электронный ресурс]. URL: <https://cucumber.io/> (дата обращения: 15.02.2026).
5. Официальная документация SpecFlow [Электронный ресурс]. URL: <https://www.specflow.com/> (дата обращения: 15.02.2026).
6. Официальная документация фреймворка Behave [Электронный ресурс]. URL: <https://behave.readthedocs.io/en/latest/> (дата обращения: 15.02.2026).
7. Stanislau Shandrokha и др. Inside Spec-Driven Development: What GitHub's Spec Kit Makes Possible For AI-assisted Engineering [Электронный ресурс]. 2026. URL: <https://www.epam.com/insights/ai/blogs/inside-spec-driven-development-what-githubspec-kit-makes-possible-for-ai-engineering> (дата обращения: 18.02.2026).
8. Марина Суворова. AI-агенты: как они устроены и что умеют [Электронный ресурс]. URL: <https://cloud.ru/blog/kak-ustroyeny-i-chto-umeuyut-ai-agenty> (дата обращения: 18.02.2026).
9. Fernando Doglio. Monolith vs Microservice Architecture: A Comparison [Электронный ресурс]. URL: <https://camunda.com/blog/2023/08/monolith-vs-microservice-architecture-comparison/> (дата обращения: 26.02.2026).

10. Кристина Тульцева. Что такое микрофронтенд [Электронный ресурс]. URL: <https://thecode.media/chto-takoe-mikrofrontend/> (дата обращения: 27.02.2026).
11. Официальная документация Apache Kafka [Электронный ресурс]. URL: <https://kafka.apache.org/> (дата обращения: 27.02.2026).
12. Официальная документация RabbitMQ [Электронный ресурс]. URL: <https://www.rabbitmq.com/> (дата обращения: 27.02.2026).
13. Официальная документация nginx [Электронный ресурс]. URL: <https://nginx.org/> (дата обращения: 27.02.2026).
14. Официальная документация Apache HTTP Server Project [Электронный ресурс]. URL: <https://httpd.apache.org/> (дата обращения: 27.02.2026).
15. Официальная документация Yandex API Gateway [Электронный ресурс]. URL: <https://yandex.cloud/ru/services/api-gateway> (дата обращения: 27.02.2026).
16. Официальная документация KrakenD [Электронный ресурс]. URL: <https://www.krakend.io/> (дата обращения: 27.02.2026).
17. Alfa-Digital [Электронный ресурс]. URL: <https://digital.alfabank.ru/techradar/> (дата обращения: 27.02.2026).
18. Т-Банк [Электронный ресурс]. URL: <https://www.tbank.ru/career/it/about/> (дата обращения: 27.02.2026).
19. Ozon Tech [Электронный ресурс]. URL: <https://ozon.tech/> (дата обращения: 27.02.2026).
20. WBTECH [Электронный ресурс]. URL: <https://wbtech.wildberries.ru/> (дата обращения: 27.02.2026).

21. Денис Цветчих. Модульный монолит вместо микросервисов [Электронный ресурс]. URL: https://www.youtube.com/watch?v=4UKjtzeQ_us (дата обращения: 27.02.2026).
22. Mark Richards. Software Architecture Patterns Second Edition. O'Reilly Media, Inc., 2022.
23. Jeffrey Palermo. Onion Architecture [Электронный ресурс]. URL: <https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/> (дата обращения: 01.03.2026).
24. Robert C. Martin. The Clean Architecture [Электронный ресурс]. URL: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html> (дата обращения: 12.01.2026).

ПРИЛОЖЕНИЕ

Репозиторий – <https://github.com/Shtskkh/Events.Backend>